

Information Retrieval and Verification Pipeline

We outline a multi-stage pipeline for robust retrieval and fact-checking. Each stage refines the input for the next. First, **text preprocessing** cleans the data; then **query expansion** bridges vocabulary gaps; next **corpus preparation** structures the knowledge base; we then **narrow the search space** in stages (lexical → semantic → reranking); finally, **entailment analysis** verifies factual consistency. The sections below describe each step in detail.

Preprocessing

Text preprocessing cleans and normalizes the raw input to reduce noise and variability. We remove irrelevant tokens (hashtags, emojis, HTML tags, punctuation, etc.) using regex-based filters, and normalize whitespace (collapse extra spaces/tabs/newlines). **Case folding** (converting all text to lowercase) unifies tokens (e.g. “Apple” and “apple” become the same). We then apply **sentence segmentation** to split paragraphs into individual sentences. Optionally, **lemmatization** (mapping words to their dictionary root) can be applied to consolidate word forms. These steps ensure the text is clean and consistently formatted before retrieval.

Proposed Solutions:

- **Case Folding:** Convert all text to lowercase (reduces token variance).
- **Remove Special Characters/OOV:** Strip out punctuation, URLs, HTML tags, and other non-alphabetic symbols.
- **Sentence Segmentation:** Split cleaned text into sentences.
- **Whitespace Normalization:** Collapse multiple spaces, tabs, and newlines into single spaces.
- **(Optional) Lemmatization:** Reduce each word to its base form using a lexical dictionary.

Technology Stack & Complexity:

Component	Implementation Strategy	Time Complexity
OOV Cleaning	nlTK corpus membership check	O(1) (Hash set lookup)
General Cleaning	Python string manipulation / Regex	O(n)

Query Expansion

To bridge vocabulary mismatches between user queries and documents, we expand the query with synonyms and related terms. A simple method is **thesaurus lookup**; for example, using WordNet to add synonyms for key words (especially nouns, verbs, and a few adjectives). This helps when the query uses different terminology than the text. In practice, we identify the Subject-Verb-Object of the query and add synonyms (e.g. “astronaut” → “cosmonaut”). More advanced (optional) methods include using model-based attribution (e.g. **Integrated Gradients** on a BERT-based model) to find important query terms and then expanding them, or **de-contextualizing** complex claims into simpler atomic statements. These expanded queries aim to retrieve a broader set of relevant passages without changing the query’s intent.

Proposed Technology:

- **WordNet Synonyms:** For each key query term (subject, verb, object), fetch synonyms from WordNet and add them to the query.
- **Controlled Expansion:** Limit added synonyms (one per POS or major term) to avoid query drift.
- **Integrated Gradient(Optional):** Utilize more sophisticated approaches like Integrated Gradient using a SLM model to identify the most significant feature in our query and then add synonyms for only those.

Corpus Preparation

The corpus (document collection) is preprocessed for efficient retrieval. We first extract text from each source while preserving structure (titles, headings, paragraphs, lists, etc.), since metadata can guide relevance. Then we split documents into smaller chunks based on their topics to allow fine-grained matching. **Semantic chunking** is recommended. In semantic chunking, we compute sentence embeddings and merge sentences into a chunk until a semantic shift is detected (topic change). The result is a set of semantically-coherent text passages (with their source context) indexed for retrieval. By chunking this way, we maintain citation-quality evidence and enable progressive narrowing of results.

Proposed Solutions:

- **Document Extraction:** Parse each source document and retain its logical structure (section headers, paragraphs).
- **Semantic Chunking:** Break documents into topical passages. For example, a recursive splitter can split on paragraph/sentence separators, and embeddings can be used to detect theme shifts between sentence groups.
- **Indexing:** Index the cleaned chunks (with metadata) in a retrieval system, preserving document IDs and locations for citation.

Reducing Search Space

Overview:

We narrow down candidates through multiple retrieval layers, improving precision before expensive modelling.

Proposed Solutions:

1. **Lexical Filtering (BM25):** Perform a keyword search using BM25 on the query against the indexed chunks. BM25 ranks passages by term frequency and inverse document frequency. We retrieve a small top fraction (e.g. ~1% of the corpus or upto 1000 docs) of hits that match the query terms. This first pass provides high-precision candidates. It is also efficient in terms of time.
2. **Semantic Similarity:** Embed the query and the BM25-filtered passages, then compute dense vector similarity (via approximate nearest neighbours). Keep only the closest passages (perhaps ~0.1–0.5% or up to 50 doc from the corpus) that are semantically relevant.
3. **Diversity Filtering (MMR)(Optional):** On the remaining set, apply Maximum Marginal Relevance (MMR) to select passages that are both relevant and non-redundant. MMR re-scores each candidate by combining its query similarity and dissimilarity to already-selected passages, ensuring diverse yet focused results.
4. **Cross-Encoder Re-Ranking:** Finally, run a cross-encoder model on the top 10–50 candidates. This model scores each pair and yields the most relevant passages. We keep only the top few (e.g. <10) passages for entailment analysis.

Entailment Analysis

The final step is to verify the factual claim (query) against the retrieved evidence. In NLP, **textual entailment** (or Natural Language Inference) checks whether a hypothesis (the claim) can be logically inferred from a premise (the evidence). We use a pretrained NLI model (RoBERTa or DeBERTa fine-tuned on an NLI dataset) to classify each (evidence, claim) pair as *entailment* (supporting), *contradiction*, or *neutral*. Model choice balances accuracy with inference speed.

Proposed solutions:

1. **NLI Inference:** Run each candidate evidence chunk paired with the claim through the NLI model. Keep evidence labeled as entailment (supporting the claim).
2. **Numeric Consistency Checks:** For claims involving numbers or quantities, perform explicit checks. For example, if evidence says “received \$10” but the claim says “\$9.50,” we flag this discrepancy as contradictory.
3. **Hedge Detection:** Identify and down-weight evidence that uses uncertain language (hedges) such as “may,” “suggest,” or “could.” Such hedged statements

indicate low confidence; claims supported only by hedges should be treated as weak.